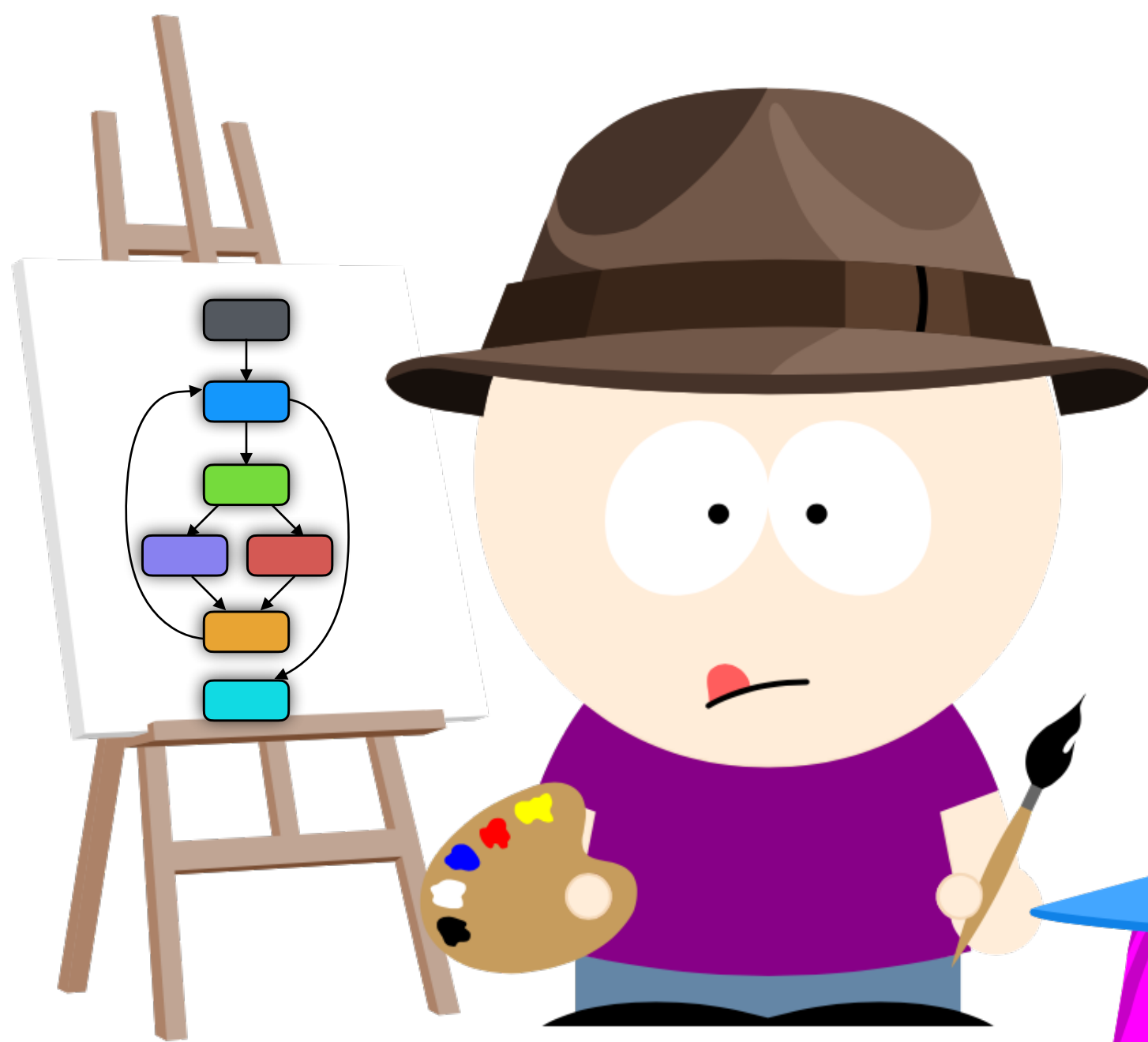




IN-CLASS EXERCISES



Version: 2026-04-06

INTERMEDIATE CODE 3 —CONTROL FLOW GRAPHS

Download exercises

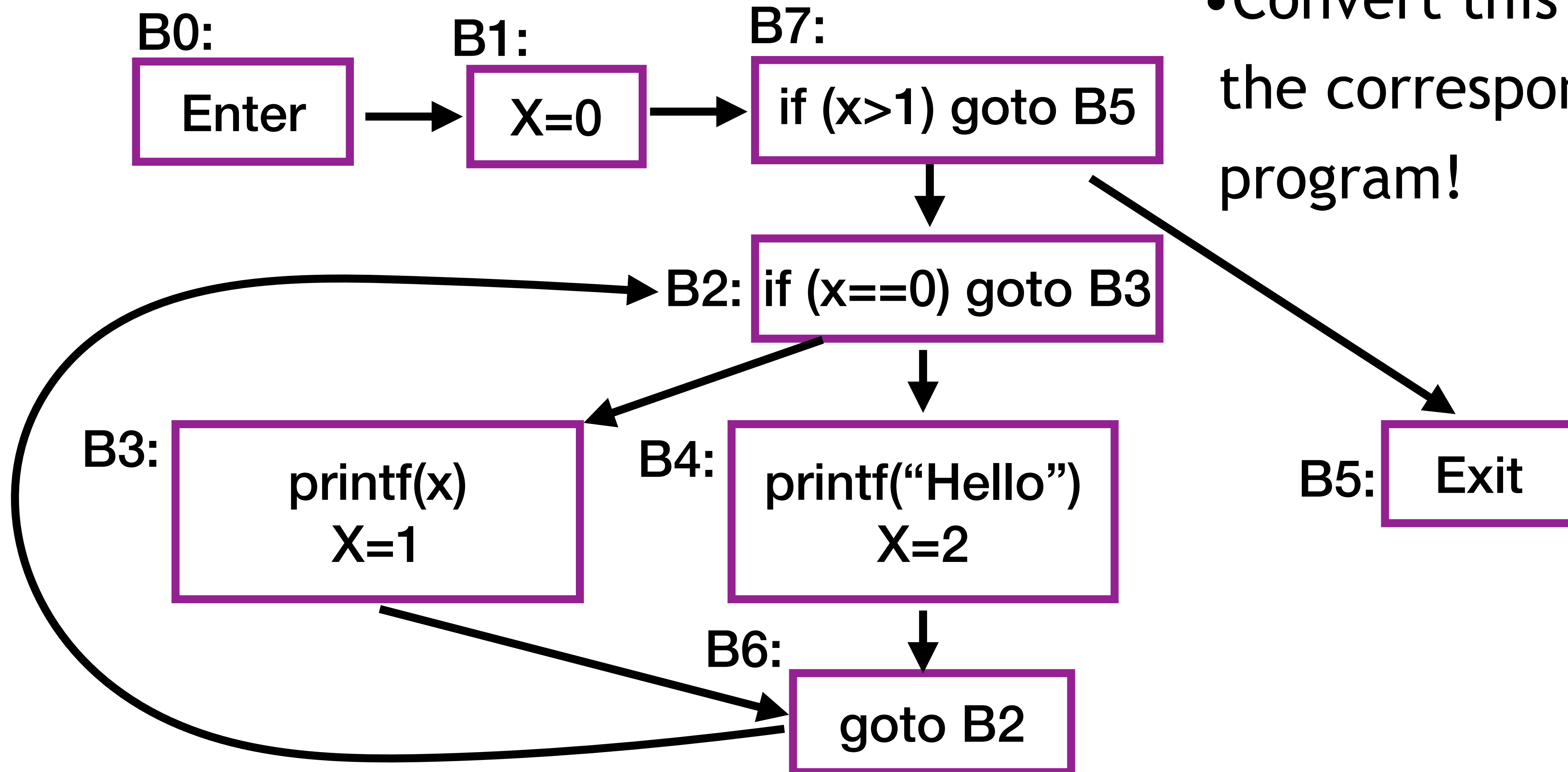
1. Inside the LigerLabs VM, open up the Firefox browser and go to <https://ligerlabs.org/compiler.html>
2. Download the file
`interm-3-exercises.zip`
3. Open up a terminal and Unzip the file

```
> unzip interm-3-exercises.zip  
> cd interm-3-exercises  
> ls
```

RECOGNIZING CFGS

Task 1: Decompile

- Convert this CFG to the corresponding C program!



Task 2: Construct CFG

- Given this sequence of 3-address statements, mark the leaders and identify the basic blocks!

```
(1)  X=20
(2)  if X >= 10 goto (8)
(3)  X=X-1
(4)  A[X] = 10
(5)  if X != 4 goto (7)
(6)  X=X-2
(7)  goto (2)
(8)  Y=X+5
```

Task 3: Construct CFG

- Following the steps of the CFG-construction algorithm, convert this function to a CFG.

```
int gcd(int x, int y) {  
    int temp;  
    while (true) {  
        if (x%y == 0) break;  
        temp = x%y;  
        x = y;  
        y = temp;  
    }  
}
```


Task 4: Pair Challenge

- Sketch a small C program with very complex control flow.
- Transform the program to a CFG.
- Switch CFGs with the partner to the left of you!
- Who can transform the program they've been given back to C the fastest?

Task 5: Exceptions

- Consider the Java program on the next slide.
- It will throw a `java.lang.ArithmeticException` exception when a division by zero occurs.
- What would a CFG for the `main` function look like?


```
class Exc {  
    void main (String[] args) {  
        int c = args.length;  
        try {  
            int i = c;  
            while (i <= c) {  
                int x = 42/i;  
                i--;  
            };  
        } catch (java.lang.ArithmeticException e) {  
            System.out.println("DIV-BY-ZERO");  
        };  
    }  
}
```



DEFINITIONS AND ALGORITHMS

2. Next, execute this algorithm to break the sequence of instructions into basic blocks (the nodes of the control-flow graph) and graph edges between these blocks:

a. Mark every instruction which can start a basic block as a leader:

- the first instruction is a leader;
- any target of a branch is a leader;
- the instruction following a conditional branch is a leader.

b. A basic block consists of the instructions from a leader up to, but not including, the next leader.

c. Add an edge $A \rightarrow B$ if A ends with a branch to B or can fall through to B.

COLLBERG.CS.ARIZONA.EDU

TIGRESS.WTF

LIGERLABS.ORG

GRAND-RE-CHALLENGE.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA